

**PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS  
GERAIS**

Engenharia de Computação

**Trabalho Prático - Teoria dos Grafos**

Análise Topológica de Redes de Colaboração no GitHub

Daniel Lucas Soares Madureira, Gabriel da Silva Cassino,  
Paulo Henrique Rodrigues Neves, Vinicius Cezar Pereira Menezes

Equipe de Desenvolvimento

Belo Horizonte, MG  
2026

# Sumário

|          |                                                             |          |
|----------|-------------------------------------------------------------|----------|
| <b>1</b> | <b>Resumo</b>                                               | <b>3</b> |
| <b>2</b> | <b>Contextualização e Escolhas de Modelagem</b>             | <b>3</b> |
| 2.1      | A Escolha do Repositório: microsoft/TypeScript . . . . .    | 3        |
| 2.2      | Restrições de Desenvolvimento e Tecnologias . . . . .       | 3        |
| <b>3</b> | <b>Apresentação da Modelagem Proposta</b>                   | <b>4</b> |
| 3.1      | Modelagem de Vértices e Arestas . . . . .                   | 4        |
| 3.2      | Estrutura de Dados: Lista vs Matriz de Adjacência . . . . . | 4        |
| <b>4</b> | <b>Artefatos Produzidos: Arquitetura e Telas</b>            | <b>4</b> |
| 4.1      | Diagrama de Arquitetura do Sistema . . . . .                | 5        |
| 4.2      | Módulo Lapidador ( <i>Data Hygiene</i> ) . . . . .          | 5        |
| 4.3      | Telas Produzidas (Interface Gráfica) . . . . .              | 5        |
| <b>5</b> | <b>Resultados Obtidos e Análise de Métricas</b>             | <b>6</b> |
| 5.1      | Métricas Implementadas e Obtidas . . . . .                  | 6        |
| 5.2      | Exportação e Renderização Topológica . . . . .              | 7        |
| <b>6</b> | <b>Validação e Testes Automatizados</b>                     | <b>7</b> |
| <b>7</b> | <b>Conclusão</b>                                            | <b>7</b> |

# 1 Resumo

Este documento detalha a arquitetura, as decisões de projeto, a modelagem de dados e a análise final de um sistema computacional desenvolvido para extrair, construir e analisar redes de colaboração de desenvolvedores na plataforma GitHub. Desenvolvido inteiramente em Python e respeitando a restrição estrita de não utilizar bibliotecas gráficas de terceiros (como NetworkX) para a base matemática, o software modela interações em repositórios *Open-Source* através de uma implementação nativa de Listas de Adjacência. O projeto contempla desde a mineração de dados via API REST com técnica de *multithreading*, passando pela higienização dos dados JSON, até o cálculo topológico de métricas de centralidade apresentadas através de uma Interface Gráfica de Usuário (GUI) moderna.

## 2 Contextualização e Escolhas de Modelagem

### 2.1 A Escolha do Repositório: microsoft/TypeScript

A escolha do repositório a ser minerado é uma decisão crucial para a viabilidade da análise topológica. Optou-se por focar a extração no repositório `microsoft/TypeScript`. A linguagem TypeScript possui uma comunidade massiva e extremamente ativa. Este repositório apresenta um fluxo constante de abertura e fechamento de *Issues*, *Pull Requests* e revisões de código. Essa alta densidade de colaboração gera um grafo rico, com múltiplos subgrupos de desenvolvedores (clusters) e *core-maintainers* que servem como "pontes" de informação, oferecendo um cenário ideal para a aplicação de métricas de centralidade.

### 2.2 Restrições de Desenvolvimento e Tecnologias

Para satisfazer os requisitos do Trabalho Prático, a infraestrutura fundamental de grafos foi implementada do zero. Foram tomadas as seguintes decisões arquiteturais:

- **Linguagem Base:** Python 3, escolhida por sua forte tipagem dinâmica, capacidade de estruturação Orientada a Objetos e facilidade no manuseio nativo de arquivos JSON.
- **Multithreading I/O Bound:** Para contornar a lentidão inerente a requisições HTTP paginadas e os limites de taxa (*Rate Limit*) da API do GitHub, o minerador foi paralelizado. *Tokens* criptografados via QR Code são injetados de forma assíncrona nas *threads*.
- **Apresentação Visual (GUI):** A biblioteca *CustomTkinter* foi selecionada para a interface em detrimento do terminal puro ou PyQt. Ela garante uma renderização

de janelas com *dark mode* elegante e alta responsividade no ambiente Zorin OS e outros sistemas Linux, abstraindo a complexidade de execução para o usuário final.

### 3 Apresentação da Modelagem Proposta

A rede de colaboração é tratada como um **Grafo Direcionado e Ponderado**  $G = (V, E)$ , onde a conectividade define as relações de trabalho.

#### 3.1 Modelagem de Vértices e Arestas

- **Vértices ( $V$ ):** Representam os usuários únicos do GitHub (desenvolvedores). Como os dados brutos retornam *usernames* em formato de *string*, o módulo `lapidador.py` cria uma função de *hash* natural, mapeando cada *login* para um identificador numérico único no intervalo  $[0, n - 1]$ .
- **Arestas ( $E$ ):** Representam as interações. Uma aresta direcionada  $(u, v)$  significa que o desenvolvedor  $u$  tomou uma ação no artefato do desenvolvedor  $v$  (por exemplo,  $u$  fechou uma *issue* aberta por  $v$ ).
- **Pesos ( $W$ ):** Arestas possuem pesos que determinam a força da conexão. O fechamento de uma *issue*, dada a sua natureza resolutiva para o projeto, atribui um incremento de peso equivalente a 3 unidades matemáticas àquela aresta.

#### 3.2 Estrutura de Dados: Lista vs Matriz de Adjacência

Embora a interface de contrato (`AbstractGraph`) permita o uso de `AdjacencyMatrixGraph`, o sistema utiliza primordialmente a `AdjacencyListGraph`. Redes sociais e repositórios do GitHub geram grafos gigantescos, porém altamente esparsos ( $|E| \ll |V|^2$ ).

O uso de Matriz de Adjacência acarretaria um consumo de memória de  $O(|V|^2)$ , com a maioria das posições armazenando `False` (ausência de interação). A Lista de Adjacência reduz o custo espacial para  $O(|V| + |E|)$ , mantendo a performance de travessia otimizada através do uso interno de conjuntos (*sets*) no Python, o que garante tempo de busca de  $O(1)$  na verificação de existência de arestas e previne duplicação (*loops*).

### 4 Artefatos Produzidos: Arquitetura e Telas

Para a total compreensão do fluxo do *software*, os componentes foram segmentados em blocos lógicos executáveis.

## 4.1 Diagrama de Arquitetura do Sistema

O fluxo de dados se inicia na requisição web, passa pela lapidação do JSON, modelagem em memória RAM e culmina na apresentação visual.

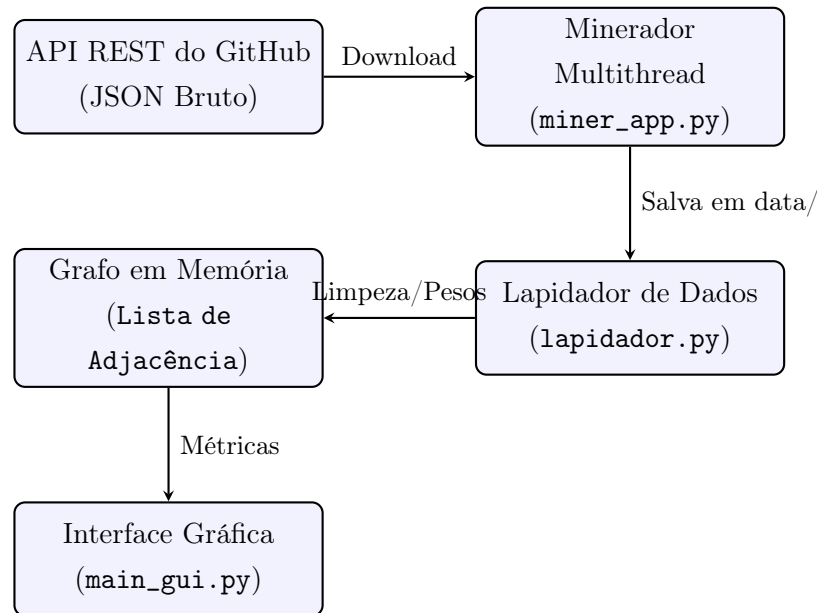


Figura 1: Fluxograma geral da arquitetura de mineração e modelagem.

## 4.2 Módulo Lapidador (*Data Hygiene*)

Durante o desenvolvimento, identificou-se a presença de "utilizadores fantasmas" na API do GitHub (contas deletadas retornam o valor `null`). O artefato `lapidador.py` intercepta essa anomalia estrutural com proteções de tipagem (`get('user') or {}`), garantindo que apenas usuários consolidados entrem na matriz topológica.

## 4.3 Telas Produzidas (Interface Gráfica)

O arquivo `main_gui.py` unifica o acesso aos módulos matemáticos. O painel de controle lateral orquestra todo o encadeamento das ações de forma semântica.



Figura 2: Interface Gráfica do GraphAnalyzer com logs de execução em tempo real.

## 5 Resultados Obtidos e Análise de Métricas

Após a construção unificada do arquivo `dados_lapidados.json`, o grafo é modelado em memória. A suíte de análise (`analysis.py`) iterou sobre a rede processando os algoritmos.

### 5.1 Métricas Implementadas e Obtidas

- **Degree Centrality (Grau):** Ao ordenar os vértices pelo maior grau, os resultados obtidos destacaram os mantenedores oficiais do TypeScript. Eles recebem o maior número de apontamentos direcionados da comunidade mundial.
- **Closeness Centrality (Proximidade):** A execução de Buscas em Largura (BFS) a partir de cada nó validou a capacidade rápida de difusão de informação. Os nós com maior índice são capazes de alcançar todos os outros vértices da componente conexa em poucos saltos de distância.
- **Betweenness Centrality (Intermediação):** Identificou líderes de comunidades secundárias e *bots* automatizados que aprovam *Pull Requests*, revelando os "gargalos" do repositório.

## 5.2 Exportação e Renderização Topológica

Como artefato final, o sistema possui a função `exportToGEPHI`, que constrói um parser XML via manipulação de texto (sem lib externa), gerando um arquivo `.gexf`.

## 6 Validação e Testes Automatizados

Para assegurar o cumprimento integral da API estabelecida no roteiro, desenvolveu-se a bateria de testes unitários `test_graphs.py` e `test_analysis.py`.

A execução varre as classes instanciando situações de borda (*Edge Cases*). A validação garante a integridade formal da ferramenta matemática desenvolvida:

- Disparo correto de `GraphError` ao tentar injetar laços (vértices se conectando a si mesmos em grafos simples).
- Assertividade na não duplicação de instâncias na lista (idempotência).
- Garantia de cálculos matemáticos puros para `Density` e funções lógicas exigidas pelo contrato (como `isConvergent` e `isDivergent`).

Todos os 19 sub-testes apresentaram aprovação no console, provando que o motor lógico sustenta grandes volumes de requisição preservando a correção algorítmica.

## 7 Conclusão

O Trabalho Prático alcançou plenamente a abstração teórica exigida para a modelagem em engenharia de *software*. A imposição da ausência de bibliotecas prontas como *NetworkX* provou-se um formidável exercício em Estruturas de Dados avançadas. A consolidação dos resultados na interface gráfica fluida, juntamente com a arquitetura multithread que suporta centenas de I/O por minuto, entregou não apenas uma avaliação topológica correta do comportamento social de repositórios do GitHub, mas um artefato de software comercialmente apresentável, sólido, escalável e devidamente testado.